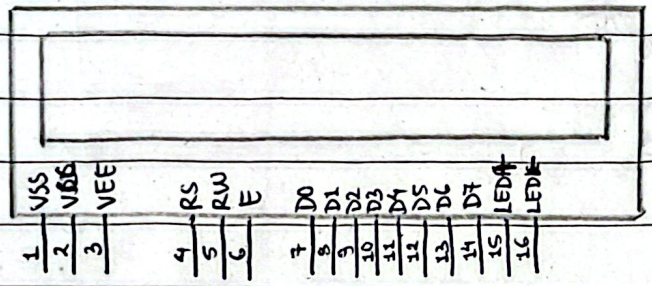
Unit-6 Peripherals & Interfacing

LCD (used to display number, text, graphics. Types of LCD: Segment LCD, Graphic LCD, Character LCD (16x2).
(display fixed patterns (alarm)) (can display image, custom characters)

LCD 16x2 Display

- Two rows each capable of displaying 16 characters
- 16 pins, LCD works by controlling liquid crystals to block or pass light. Forms character using dot matrix



NO.	Pin	Function
1	VSS	Ground
2	~~VDD~~ VCC	+5V supply
3	VEE	Display contrast control (controlled by varying voltage by potentiometer GND → Maxm contrast)
4	RS	Register select (0 = command Register, 1 = Data Register)
5	RW	Read/write (0 = write, 1 = Read)
6	E	Enable
7-14	D0-D7	Data Pins (Parallel data/command interface; 4 bit mode ⇒ D4-D7 used)
15	LED+	+ve supply for backlight
16	LED-	-ve supply for backlight

Register select

Command Register: RS = 0 : The data sent to D0-D7 is interpreted as command
Data Register: RS = 1 : The data sent to is interpreted as content to be displayed

Enable
This pin is used to tell LCD to read data from data pins.

RW
RW = 0 : Data written to LCD
RW = 1 : Data read from LCD

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

1 = 00000001
1<<5 = 00100000
~(1<<5) = 11011111

Date
Page

Q) Write an AVR C program to display the message "Hello, world!" on a 16x2 LCD. ~~using 4-bit mode.~~

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>

#define LCD PORTA
#define EN 7
#define RW 6
#define RS 5


void lcdcmd(unsigned char cmd)
{
    PORTD &= ~(1<<RS);          //RS=0 for command
    PORTD &= ~(1<<RW);          //RW=0 for write
    LCD = cmd;
    PORTD |= (1<<EN);           //Enable Pulse
    _delay_ms(1);
    PORTD &= ~(1<<EN);
    _delay_ms(3);
}

void lcddata(unsigned char data)
{
    PORTD |= (1<<RS);           //RS=1 for data
    PORTD &= ~(1<<RW);          //RW=0 for write
    LCD = data;
```

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

```
    PORTD |= (1<<EN);
    _delay_ms(1);
    PORTD &= ~(1<<EN);
    _delay_ms(3);
}

void lcd_init()
{
    DDRA = 0xFF;          //define output port
    DDRD = 0xFF;          //define RS, EN, RW pin as o/p
    PORTD &= ~(1<<EN);    //Initialize EN=0

    lcdcmd(0x38);         //Initialization of 16x2 LCD in 8 bit mode
    lcdcmd(0x0C);         //display on cursor off.
    lcdcmd(0x06);
    lcdcmd(0x01);
    lcdcmd(0x80);
}

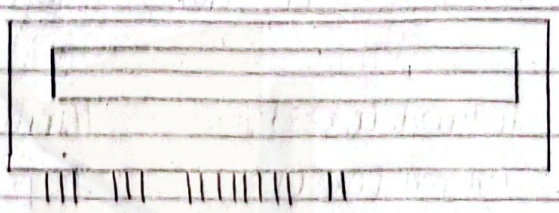
void lcd_print(char *str)
{
    unsigned char i=0;
    while(str[i]!=0)
    {
        lcddata(str[i]);
        i++;
    }
}
```

notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

```
int main()
{
    lcd_init();
    lcd_print("Hello Arpan");
    lcdcmd(0x80);        //Go to 2nd line
    lcd_print("second line");
    return 0; while(1);
}
```



notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

adhikari-arpan/BECE_Notes

Peripherals are external components that allow microcontroller to sense, process & act upon i/p & o/p.
Sensors allow system to gather information about the environment.

	Analog sensor	Digital sensor
Output	Analog output which is continuous voltage between 0 to 5V mostly	Binary output (either HIGH or LOW corresponding to 1 or 0)
signal type	smooth	Discrete
ADC Required	Yes	No
Accuracy	Higher (detailed values)	Lower (basic detection)
Complexity	More complex	Easier to use
Example	LM35 (Temp^r sensor), LDR	DHT11 (Temp^r and Humidity sensor), PIR (Motion sensor)

LM35 (Temperature sensor)

- Analog sensor used to measure temperature in °C.

Working:- LM35 gives analog voltage proportional to 10mV per °C.

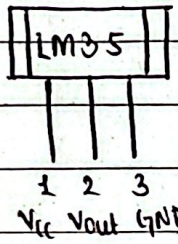
e.g. 0°C → 0V

25°C → 0.25V

So, Temperature (°C) = Output voltage × 100

$10^{mV} = \frac{\text{o/p voltage}}{\text{Temp}^r}$

$\text{Temp}^r = \frac{\text{o/p voltage}}{10mv}$



1 → Vcc
2 → Vout
3 → GND

notes.arpanadhikari7.com.np

- Linear output (easy calculation)
- No calibration needed
- Low power consumption
- High accuracy (±0.5°C)
- wide Range: -55°C to 150°C
- Works with 4V-30V supply

Interface with Atmega 32:

LM35 Pin	Atmega 32
Vcc	5V
Vout	ADC
GND	GND

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Q) Program to interface temperature sensor

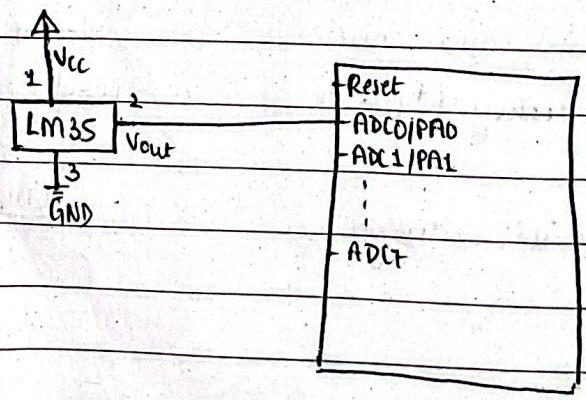
```
#define F_CPU 8000000UL
#include <avr/io.h>

void adc_init(){
    DDRA = 0x00;          //Input
    ADMUX = 0xE0;
    ADCSRA = 0x87;
}

unsigned char adc_read(){
    ADCSRA |= (1<<ADSC);      //start conversion
    while(ADCSRA & (1<<ADSC)); //wait for end of conversion
    return ADCH;
}

int main(void){
    DDRB = 0xFF ;         //Output
    adc_init();
    while(1){
        PORTB = adc_read();
    }
}
```

notes.arpanadhikari7.com.np



Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Assembly program:

```
.include "m16def.inc"

ldi r16, 0x00          (Load Immediate)
out DDRA, r16

ldi r16, 0xE0
out ADMUX, r16
ldi r16, 0x87
out ADCSRA, r16
ldi r16, 0xFF
out DDRB, r16

main_loop:

  sbi ADCSRA, ADSC          (set bit)
  wait_adc:
      sbic ADCSRA, ADSC
      rjmp wait_adc          ;relative jump

    in r16, ADCH
    out PORTB, r16

  rjmp main_loop
```

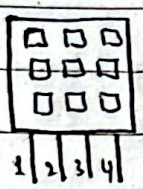
notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Temperature & Humidity Sensor (DHT11), DH22

- Measure temp^r and amount of water vapor present in air
- Digital sensor
- Use single ~~u~~digital signal to provide data for both temp^r and humidity
- Need a pull-up resistor to ensure proper communication.
- Cheap, easy to use, popular for Arduino, AVR project



4 pins: 1 → Vcc
2 → Data (connect to pull up resistor)
3 → NC (Not connected)
4 → GND

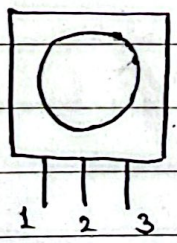
Range: 0-50°C (Temp^r)
20-90% RH (Humidity)

Supply: 3.3-5V

Update rate: ~1 Hz (Read once per second)

PIR Sensor / (Passive Infrared) Motion sensor

- Digital sensor
- Detects motion by sensing infrared radiation emitted by humans or animals
- "Passive" means it doesn't emit anything, it just detects IR from objects.
- Common for security systems, automatic lights, alarms



3 pins:
1 → Vcc
2 → OUT
3 → GND

Interface:	Sensor Pin	AVR Pin	
	Vcc	+5V	Input pin → read digital value
	OUT	PD0	If HIGH → motion detect → trigger LED, buzzer or alert
	GND	GND	If LOW → no motion

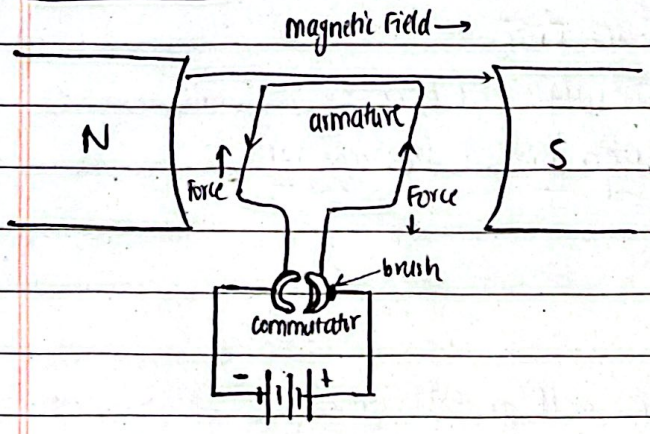
Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Actuator Interfacing

Actuator are device that perform actions in response to command from microcontroller.

DC motor :

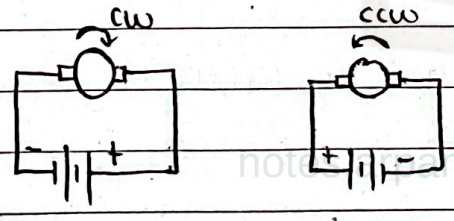


setup as given. When current is passed in coil, by using Fleming Left hand rule, force is exerted as shown in fig.
cause armature to rotate on clockwise direction.

Controlling DC Motor includes: (i) controlling the motor's speed
(ii) controlling motor direction of rotation (cw or ccw)

(i) controlling direction

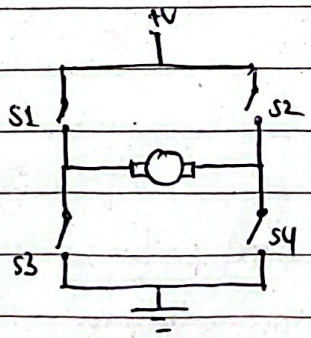
~~1. Simple method~~ (by polarity)



Unidirectional control → rotates in one direction based on polarity

bidirectional control → Reversing direction
(i) Relays
(ii) H-bridge circuit

H-bridge



CW → S1 & S4
CCW → S2 & S3
All off → stop

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Date
Page

Motor driver IC (L298N Motor driver)
- Dual H-bridge IC
- can control 2 DC motors or 1 stepper motor
- Use Motor drives because : (don't connect directly to μc)
  - Back EMF
  - Insufficient power
  - High current damage to μc

(ii) controlling speed

Method 1 : Variable voltage (Increase voltage → Increase speed, Not practical) More loss.

Method 2 : Pulse width Modulation (PWM)

PWM for controlling brightness, speed

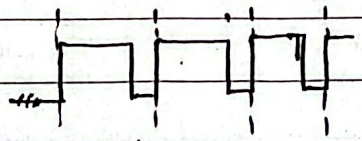
PWM is a technique used to generate analog like output using digital signals, so that analog device can operate using digital
PWM varies how long the signal stays HIGH vs LOW.

Concept : Duty cycle : percentage of how long signal stay HIGH in a cycle.

$$\text{Duty cycle} = \frac{T_{ON}}{T_{ON} + T_{OFF}} \times 100\%$$

notes.arpanadhikari7.com.np

T_{ON} = High
T_{OFF} = Low



75% duty cycle

PWM working :
- Signal rapidly switches ON and OFF
- Frequency is so high (human eye can't detect flicker)
- Device works on avg. voltage $V_{avg.}$ = duty cycle × V_{man}

Switching frequency → freq. of switch

Scanned with CamScanner

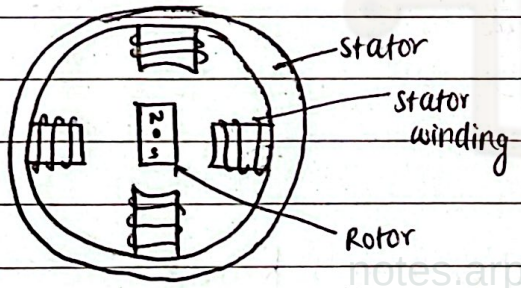
Available on notes.arpanadhikari7.com.np

PWM pin available in arduino with (~) mark.

Controlling brightness:
- LED bulb turns ON and OFF very fast.
- Human eye can't detect flicker. - Works on avg. voltage
  Low duty cycle → DIM light
  High duty cycle → Bright light

Controlling speed:
- Controls speed by varying average voltage supplied to motor.
- Instead of continuous voltage, rapidly switching voltage.
- Speed depends on duty cycle. $V_{avg.} = \text{duty cycle} \times V_{max}$.
- At Off; keeps moving due to inertia. So, act like working with avg. voltage (less speed)

Stepper Motor
- Convert electrical signal to discrete mechanical steps.
- Unlike DC motor, stepper move in fixed step, not continuous rotation. Provide precise position control.



1. Current flows through stator winding → creates magnetic field.
2. Rotor aligns with magnetic field.
3. Next coil energized. → rotor move to next position.
4. Repeat steps.

notes.arpanadhikari7.com.np

motor driver circuit used for speed control, direction control

Stepper Motor driving modes:
1. Wave Drive (Single coil Excitation) — A → B → C → D → Repeat
2. Full step drive (Two coil Excitation) — A+B → B+C → C+D → D+A → Repeat
3. Half Step drive (Combination) — A → A+B → B → B+C → C → C+D → D → D+A → Repeat
   (Rotor move in smaller step, more precision)

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Step angle = Basic movement of stepper (angle in each step)

$$\text{step angle} = \frac{N_s - N_r}{N_s \times N_r} = \frac{360}{N \times P}$$

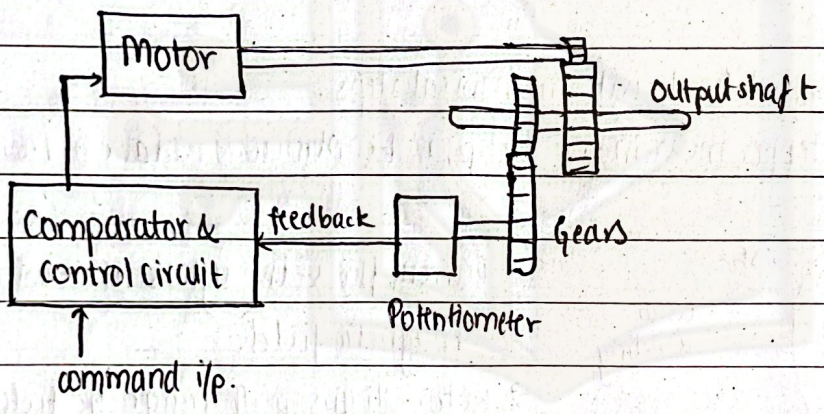
N_s → No. of stator pole
N_r → No. of rotor pole

N → No. of teeth of rotor, P → No. of phases.

$$\text{Steps per revolution} = \frac{360^\circ}{\text{step angle}}$$

Servo Motor : standard servo : 0 to 180° rotation

- A servo motor is a rotary actuator that provides precise control of position, speed and sometimes torque.
- Used to complete task in precise way
- Two types of servo motor: (i) AC servo motor (ii) DC servo motor



- Servo Motor works using closed loop control system with feedback.

Components: 1. Motor (AC or DC)
2. Feedback Device (Potentiometer or Encoder)
3. Control circuit (Brain of system, compares desired & current condition)
4. Drive circuit (Provides power to motor, controls direction & speed)
5. Gear Box

Scanned with CamScanner
Available on notes.arpanadhikari7.com.np

Working (Closed loop system):
- We give input (desired angle) by PWM.
- Controller read current position from sensor.
- Compare value. Error = Desired Position − Current Position
- If error exists, motor rotates.
- Feedback updates continuously.
- Motor stop when error = 0.

{Continuous correction ⇒ closed loop control}

- PWM in servo is used to control angle. (Not speed but angle).

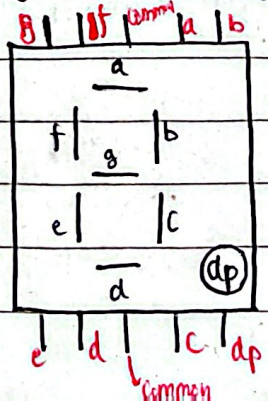
E.g ~~1ms~~ Pulse width: ~1ms = 0°
~1.5ms = 90°

- Used in robotic arms, camera positioning

Interface		
Red wire	V_{cc}	MC 5V
Black wire	GND	GND
Yellow/orange	control signal	PWM pin

Display Interfacing (Display data in human readable form)

Seven segment Display
- Electronic device used to represent number & limited alphabetic characters.
- Consists of seven LED segments (labeled 'a' to 'g'), with an additional dot segment (dp) for decimal points.
- Characters displayed by selectively illuminating these segments.
- Types: 1. Common Cathode (Common pin = GND, Illumination by suppling (logic 1) V)
  2. Common Anode (Common pin = V_{cc}, supply (logic 0) for Illumination)



Scanned with CamScanner
Available on notes.arpanadhikari7.com.np

adhikari-arpan/BECE_Notes



g	f	e	d	c	b	a
0	1	1	1	1	1	1

Date
Page

Q) Display from 0 to 9.

```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>

int main(void){
    DDRC = 0xFF;        //Set PortC as o/p
    PORTC = 0x00;       //Turn off all segments initially

    char array[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07,
                    0x7F, 0x6F};
                    //Hex values from 0 to 9

    while(1){
        for(int i=0; i<10; i++){
            PORTC = array[i];
            _delay_ms(1000);
        }
    }
    return 0;
}
```

notes.arpanadhikari7.com.np

increment on button press:

```
#define SWITCH PD0
#define SWITCH_PIN PIND

if SWITCH_PIN
DDRC = 0xFF;
PORTC = 0x00; //Initial 0
```

```
DDRD &= ~(1<<SWITCH);
PORTD = (1<<SWITCH);
//Pull up रेजिस्टर enable
while
char array[] = {...}
int count=0;
```

```
while(0){
if((!(SWITCH_PIN & (1<<SWITCH))))
{
count++;
if(count>9){
    count=0;
PORTC = array[count];
}
```

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np